

CogniStream — Technical Review Guide

Developer Productivity Analytics Pipeline (Weeks 1 to 3)

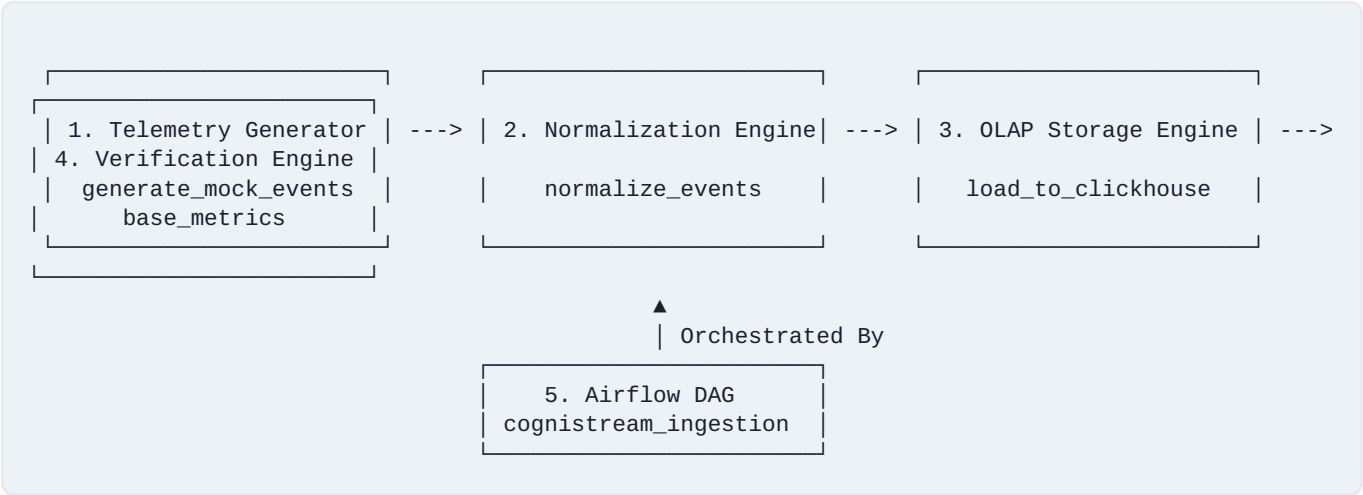
- Apache Airflow
- Polars Engine
- chdb (ClickHouse)
- Docker Parity

1. Executive Summary

CogniStream is an end-to-end developer productivity analytics engine designed to capture, process, and analyze developer telemetry data across multiple operational platforms (GitHub, Slack, IDE focus tracking, and Jira tickets). The goal is to measure developer flow states, cognitive load, and context switching patterns accurately.

By Week 3, the project successfully establishes a containerized, automated ingestion and persistent analytical storage pipeline. Raw events are generated, normalized using Rust-backed Polars DataFrames, persisted in a local vector-optimized ClickHouse instance via `chdb`, and scheduled daily using Apache Airflow.

2. System Architecture & Components



Core Component Breakdown:

- **Telemetry Generator** (`generate_mock_events.py`): Simulates realistic developer activity streams (e.g., PR reviews, Slack messages, IDE focus sessions, Jira ticket updates) and outputs standardized event records.
- **Data Normalization Engine** (`normalize_events.py`): Leverages **Polars** (a high-performance, multithreaded DataFrame library) to clean raw payloads, impute missing values, enforce type safety, and standardize event timestamps to UTC.
- **Persistent Storage Layer** (`load_to_clickhouse.py`): Utilizes `chdb` (an in-process vectorized ClickHouse engine) to insert normalized data into persistent columnar tables mapped to local disk storage (`/opt/airflow/chdb_data`).
- **Verification Script** (`base_metrics.py`): Queries the embedded ClickHouse instance using SQL to extract basic developer interaction counts and aggregate metrics to verify data integrity.
- **Orchestration DAG** (`cognistream_ingestion_dag.py`): Apache Airflow pipeline scheduled daily (`0 9 * * *`) managing task dependencies: `generate_events >> normalize_data >> load_clickhouse` .

3. Step-by-Step Data Pipeline Lifecycle

Phase	Component Script	Technology Stack	Primary Action & Output
1. Ingestion	<code>generate_mock_events.py</code>	Python, JSON	Generates raw developer telemetry payloads across PR, Slack, IDE, and Jira channels.
2. Processing	<code>normalize_events.py</code>	Polars DataFrame	Standardizes timestamps, casts schemas, handles null values, and produces normalized Parquet/CSV output.
3. Persistence	<code>load_to_clickhouse.py</code>	chdb (ClickHouse)	Performs vectorized insertion into ClickHouse tables inside volume-mounted storage <code>/opt/airflow/chdb_data</code> .
4. Validation	<code>base_metrics.py</code>	chdb SQL Engine	Executes analytical queries to verify row counts, event distributions, and metric accuracy.
5. Scheduling	<code>cognistream_ingestion_dag.py</code>	Apache Airflow	Orchestrates daily execution workflow with 5-minute retry intervals and error handling.

4. Mid-Review Viva Questions & Answers

Q1: Why choose Polars instead of Pandas for data normalization?

Answer: Polars is written in Rust and uses Apache Arrow memory formatting. It executes operations in parallel across multiple CPU cores with lazy evaluation and memory efficiency. Compared to Pandas, it processes columnar telemetry data significantly faster with much lower RAM overhead.

Q2: Why use `chdb` rather than PostgreSQL or MySQL?

Answer: `chdb` gives us an in-process, zero-setup ClickHouse engine designed specifically for OLAP (Online Analytical Processing). Columnar databases compress repetitive event logs efficiently and execute aggregation queries (e.g., calculating developer focus duration over millions of rows) orders of magnitude faster than row-based OLTP databases like PostgreSQL.

Q3: How is persistent storage guaranteed inside Docker?

Answer: ClickHouse data is configured to write to `/opt/airflow/chdb_data`, which is volume-mapped to the host filesystem. This ensures that database records and state persist across container restarts and builds.

5. Terminal Verification & Git Commands

End-to-End Execution Command inside Docker:

```
docker exec -it cognistream-airflow bash -c "python3 /opt/airflow/generate_mock_events.py &&
python3 /opt/airflow/normalize_events.py && python3 /opt/airflow/load_to_clickhouse.py &&
python3 /opt/airflow/base_metrics.py"
```

Git Commit & Push Commands:

```
cd /Users/harishkaruppuchamy/Downloads/COGNISTREAM
git add .
git commit -m "feat: verify end-to-end ingestion pipeline and chdb clickhouse persistence"
git push origin Harish-analytical-storage-pipeline
```